CATEGORY C

Part A — Coding Problems

Problem 1: Garden Plot Area Report

TASK

A community garden rents out plots of three shapes: circular, rectangular, and triangular. Every plot has an owner, and every plot's area is calculated with a different formula. The garden office needs the area of each plot and the total area rented.

Adding a new plot shape later should not require changes to the code that prints the report.

INPUT / OUTPUT

Input: The first line contains N, the number of plots. Each of the next N lines contains the shape, the owner's name (one word), and the measurements: `CIRCLE Owner radius`, `RECTANGLE Owner length width`, or `TRIANGLE Owner base height`.

Output: For each plot, print `Owner (SHAPE): area`. Finally, print `Total Area: total`. All values are formatted to two decimal places.

BUSINESS RULES

- Circle area = π × radius × radius (use the language's built-in value of π).
- Rectangle area = length × width.
- Triangle area = ½ × base × height.

EXAMPLE

Sample Input

```
3
CIRCLE Asha 5
RECTANGLE Ravi 4 6
TRIANGLE Neha 10 3
```

Expected Output

```
Asha (CIRCLE): 78.54
Ravi (RECTANGLE): 24.00
Neha (TRIANGLE): 15.00
Total Area: 117.54
```

HINT

> *Consider how to represent different plot shapes that share an owner but have different area calculation logic. Think about a common base type for all plots. The design should allow for adding new plot shapes without modifying existing reporting code.*

STARTER CODE

```
// Implement: Garden Plot Area Report
```

Problem 2: Weekly Staff Pay

TASK

A small company pays three kinds of staff every week: full-time staff receive a fixed weekly salary, hourly staff are paid by the hour with overtime, and interns receive a fixed stipend. Every staff member has a name, and the payroll screen must list each person's pay and the total.

A generic "staff member" with no pay rule must never be created.

INPUT / OUTPUT

Input: The first line contains N. Each of the next N lines is one of: `FULLTIME Name weeklySalary`, `HOURLY Name hours rate`, or `INTERN Name stipend`.

Output: For each person, print `Name: pay`. Finally, print `Total Payroll: total`, formatted to two decimal places.

BUSINESS RULES

- Full-time staff are paid their fixed weekly salary.
- Hourly staff are paid `rate` for the first 40 hours and 1.5 × `rate` for every hour above 40.
- Interns are paid their fixed stipend.

EXAMPLE

Sample Input

```
3
FULLTIME Asha 12000
HOURLY Ravi 45 200
INTERN Neha 5000
```

Expected Output

```
Asha: 12000.00
Ravi: 9500.00
Neha: 5000.00
Total Payroll: 26500.00
```

HINT

> *How can you handle different pay calculation rules for various staff types while sharing common staff details? Consider a base type for all staff that prevents direct instantiation but forces subclasses to define their pay calculation.*

STARTER CODE

```
// Implement: Weekly Staff Pay
```

Problem 3: Library Late Fine Counter

TASK

A library charges a fine when borrowed items are returned late. Books, DVDs, and magazines each follow a different fine rule. Every item has a title and a number of days late. The library needs the fine for each item and the total fines collected.

INPUT / OUTPUT

Input: The first line contains N. Each of the next N lines contains `TYPE Title daysLate`, where TYPE is BOOK, DVD, or MAGAZINE and Title is one word.

Output: For each item, print `Title: fine`. Finally, print `Total Fines: total`, formatted to two decimal places.

BUSINESS RULES

- Books: 2 per day late.
- DVDs: 5 per day late, up to a maximum of 50.
- Magazines: 1 per day late.

EXAMPLE

Sample Input

```
3
BOOK Algebra 4
DVD Inception 12
MAGAZINE Sports 3
```

Expected Output

```
Algebra: 8.00
Inception: 50.00
Sports: 3.00
Total Fines: 61.00
```

HINT

> *Think about how to define a common structure for library items while allowing each item type to implement its specific fine calculation. Consider a base type that mandates a fine calculation method.*

STARTER CODE

```
// Implement: Library Late Fine Counter
```

Problem 4: Electricity Connection Billing

TASK

The electricity board issues monthly bills to homes, shops, and factories. Every connection has a number of units used, but each connection type calculates its bill differently. The board needs each connection's bill and the total amount billed.

INPUT / OUTPUT

Input: The first line contains N. Each of the next N lines contains `TYPE units`, where TYPE is HOME, SHOP, or FACTORY.

Output: For each connection, print `TYPE: bill`. Finally, print `Total: total`, formatted to two decimal places.

BUSINESS RULES

- Home: 5 per unit for the first 100 units and 7 per unit after that.
- Shop: 8 per unit plus a fixed charge of 100.
- Factory: 6 per unit, with a minimum bill of 1000.

EXAMPLE

Sample Input

```
3
HOME 150
SHOP 90
FACTORY 120
```

Expected Output

```
HOME: 850.00
SHOP: 820.00
FACTORY: 1000.00
Total: 2670.00
```

HINT

> *How can you model different electricity connection types that share the concept of units but have distinct billing logic? Consider a base type for connections that defines the common units property and an abstract method for calculating the bill.*

STARTER CODE

```
// Implement: Electricity Connection Billing
```

Problem 5: Travel Booking with a Common Fee

TASK

A travel website sells bus, train, and flight tickets. The fare depends on the travel mode and the distance, but every booking — whatever the mode — also adds the same booking fee of 50. The fee must be written in one place only.

If the booking fee changes, only one line of code should need to change.

INPUT / OUTPUT

Input: The first line contains N. Each of the next N lines contains `MODE distanceKm`, where MODE is BUS, TRAIN, or FLIGHT.

Output: For each booking, print `MODE: total`, formatted to two decimal places.

BUSINESS RULES

- Bus fare: 2 per km.
- Train fare: 1.5 per km.
- Flight fare: 2500 plus 4 per km.
- Every booking adds a booking fee of 50 to the fare.

EXAMPLE

Sample Input

```
3
BUS 200
TRAIN 300
FLIGHT 500
```

Expected Output

```
BUS: 450.00
TRAIN: 500.00
FLIGHT: 4550.00
```

HINT

> *Consider how to implement a shared booking fee across different travel modes while allowing each mode to calculate its base fare uniquely. An abstract base class could handle the common fee, with subclasses implementing the mode-specific fare calculation.*

STARTER CODE

```
// Implement: Travel Booking with a Common Fee
```

Part B — Quiz

Choose the single best answer for each question.

Q1. Which of the following best describes abstraction in object-oriented programming?

A. Showing all internal details of an object to the user
B. Protecting data inside an object from unauthorized access
C. Hiding how an object performs its operations and showing only what it does
D. Allowing a class to inherit from multiple parent classes

Q2. Which statement about an abstract class in Java is true?

A. It must contain at least one abstract method
B. It can be instantiated if it has no abstract methods
C. It cannot be instantiated directly
D. It can extend multiple other classes

Q3. If a concrete class extends an abstract class but fails to implement an inherited abstract method, what is the consequence?

A. Java automatically provides an empty implementation for the method
B. The class must be declared abstract, or the code will not compile
C. A runtime error will occur only when the unimplemented method is called
D. The compiler will issue a warning but still allow compilation

Q4. Which of the following modifiers is NOT allowed for an abstract method in Java?

A. protected
B. public
C. private
D. default (package-private)

Q5. Fields declared within a Java interface are implicitly:

A. private and static
B. protected and final
C. public, static, and final
D. public and instance-specific

Q6. How many interfaces can a single Java class implement?

A. Only one interface at a time
B. At most two interfaces
C. Any number of interfaces
D. None, if the class already extends another class

Q7. If a class implements an interface, what access modifier must it use for the implemented methods?

A. default (package-private)
B. private
C. protected
D. public

Q8. Why does Java allow a class to implement multiple interfaces but only extend a single class?

A. Implementing interfaces makes the code run faster
B. Classes are unable to contain any methods
C. Interfaces carry no instance state, avoiding the 'diamond problem' ambiguity with conflicting data inheritance
D. Interfaces are compiled independently from classes

Q9. What happens if a class implements two interfaces that both provide a default method with the same signature, and the class does not override this method?

A. The method from the first interface listed in the `implements` clause is used
B. The method from the last interface listed in the `implements` clause is used
C. A runtime exception occurs when the method is called
D. A compile-time error occurs until the class overrides the method

Q10. Which of the following is true regarding constructors in Java?

A. Concrete classes cannot have constructors
B. Abstract classes can have constructors that run when a subclass object is created
C. Interfaces must have at least one constructor
D. Constructors in abstract classes are used to create objects of the abstract class directly

Part C — Concept Questions

Answer each question in a few sentences, giving reasons for your answer.

Question 1

Explain abstraction in your own words. Provide a real-life example, different from those mentioned in the material, detailing what is shown to the user and what details are hidden.

Question 2

Describe the primary difference between abstraction and encapsulation. Explain how these two concepts work together within a single class to manage complexity and data protection.

Question 3

An abstract class cannot be instantiated, yet it can define a constructor. Explain why an abstract class might have a constructor and describe when this constructor is executed.

Question 4

Explain why an abstract method cannot be declared with the private, static, or final modifiers. Provide a reason for each of these restrictions.

Question 5

What is the 'diamond problem' in the context of inheritance? Explain how Java's design, allowing a class to implement multiple interfaces but extend only one class, addresses or avoids this problem.